

# plnt

---

The orchestration runtime for small-team agent workflows on Kubernetes

Sagar Bonde · [github.com/sagarb27](https://github.com/sagarb27)

2026-07-15

---

- [plnt](#)
- [1. Problem](#)
  - [The situation on the ground](#)
  - [The engineering cost of one AI feature](#)
  - [The universal pattern](#)
- [2. Solution](#)
  - [The three layers](#)
  - [One-command deploy](#)
- [3. Playground](#)
- [4. Why now](#)
  - [Why this team, why this month](#)
- [5. Market context](#)
  - [Direct competitors](#)
  - [Adjacent tools \(complementary, not competitive\)](#)
  - [What we do that nobody else does](#)
- [6. Product](#)
  - [What ships today \(v0.1\)](#)
  - [What's next \(v0.2 — 2 weeks\)](#)
  - [v0.3-v1.0 roadmap](#)
- [7. Architecture](#)
  - [Layered view](#)
  - [Data model](#)
- [8. Deploy story](#)
  - [Single-tenant DigitalOcean today](#)
  - [Multi-tenant later](#)
  - [Alternate paths](#)
- [9. Security posture](#)
- [10. Business model](#)
  - [Layered pricing](#)
  - [Reference consumer as flywheel](#)
- [11. Team](#)
- [12. Ask](#)
  - [Design partners](#)
  - [Seed capital \(when ready\)](#)

- [13. Links](#)
- [One sentence](#)

# plnt

---

## Orchestration runtime for small-team agent workflows on Kubernetes.

Live surfaces:

- Website: [plnt.work](https://plnt.work)
  - Playground: [plnt.work/playground](https://plnt.work/playground)
  - Code: [github.com/plnt-work](https://github.com/plnt-work) — four repos, all under one org.
- 

# 1. Problem

---

## The situation on the ground

Take one Google Business owner. A dentist, a plumber, a small hotel — pick one. Their day-to-day includes:

- Replying to Google reviews (there are 20 of them and they all sound similar).
- Posting weekly Google Business updates (they forget half the weeks).
- Triaging booking inquiries that come in through Google Maps (some are spam, some are real).

They pay for a SaaS product to help. That SaaS product wants to ship an AI feature for each of those jobs.

## The engineering cost of one AI feature

The SaaS engineer building “AI-drafted review reply” has to write, from scratch:

1. A prompt. And a prompt versioning system, because prompts change weekly.
2. A retry loop. Because the model provider will 503 on you.
3. A rate limit. Because the customer plan has budget caps.
4. A container image. Because the model runs in Kubernetes.
5. A Helm chart. Because someone has to define the Deployment / Service / HPA.
6. A canary rollout. Because you cannot ship a new prompt to 100% of traffic the day you write it.
7. A rollback path. Because the canary will sometimes fail.



Bottom to top:

## Layer 1 — plnt (runtime)

An opinionated Helm-native runtime. Reads a `WorkflowRun` CRD, runs a durable Temporal saga:

1. `pull_spec` — pull the recipe from the registry, hash-verify.
2. `resolve_backend` — validate the target cluster + GPU class exists.
3. `helm_install_canary` — install the workflow chart at 5% traffic.
4. `run_smoke_test` — N invocations through the canary; assert p95 gate.
5. `promote_or_rollback` — pass -> 100% traffic; fail -> `helm rollback`.

Every step has its own retry policy. Non-retryable error types ( `SpecInvalid` , `BackendUnavailable` ) short-circuit obvious dead-ends. All state is Temporal-tracked; restarts resume from the last completed step.

## Layer 2 — microagents (registry)

An open S3-hosted registry of workflow recipes. Each recipe is a self-describing bundle:

```
s3://microagents/review-responder/1.1.0/
workflow.yaml      # step DAG
prompts/           # per-step system prompts
tools.yaml         # tool bindings (HTTP endpoints, function schemas)
README.md
LICENSE
```

Recipes are versioned, referenced by `name@version` , and integrity-verified on pull. S3 today, OCI-registry backend on the v0.4 roadmap.

## Layer 3 — google-business (reference consumer)

The Google Business SaaS product built on plnt. It:

- Presents the review / post / booking-triage UI to end users.
- Pulls the relevant microagents recipe on customer signup.
- Calls plnt's runtime API to invoke workflows on the shared cluster.

This is the “does this stack actually let you ship a product?” proof.

## One-command deploy

The whole flow is triggered by a single `kubectl apply` :

```

apiVersion: plnt.work/v1
kind: WorkflowRun
metadata:
  name: review-responder-prod
spec:
  workflow:
    ref: review-responder@1.1.0
    registry: s3://microagents
  backend:
    cluster: gpu-cluster-01
    gpuClass: nvidia.com/h100
    gpuCount: 2
    replicas: { min: 1, max: 4 }
  canary:
    trafficPercent: 5
    smokeTest: { invocations: 10, p95BudgetMs: 2500 }

```

```
kubectl apply -f review-responder.yaml
```

15 seconds later on a warm kind cluster, or ~3 minutes on a cold real GPU cluster, that workflow is serving traffic through the playground API. The end-user SaaS starts hitting it via its OpenAI-compatible endpoint.

### 3. Playground

`playground.plnt.work` is the live proof surface. It is a **micro-agent orchestration backend** that runs on real Kubernetes with real GPU nodes.

A visitor lands, sees the model / workflow catalog, picks one, sends a prompt, watches tokens stream in. Under the hood: the request hits the playground FastAPI, which routes via `RuntimeAdapter` to the currently- promoted canary of whichever workflow was invoked, which forwards the request to the underlying GPU pod (vLLM / TGI / SGLang) with the recipe's prompt + tool bindings applied.

The visitor sees a chat panel. The evaluator sees the runtime working.

## 4. Why now

---

### The three assumptions we're betting on:

1. **Every product team is going to build AI features.** Signal: it is already happening. Every SaaS incumbent has an AI announcement per quarter. Every early-stage founder is prototyping with the OpenAI SDK.
2. **They are going to be stuck at the deploy layer.** Signal: the OSS runtime ecosystem (vLLM, TGI, SGLang, llama.cpp-server) has converged on OpenAI wire compatibility in the last 18 months. The contract is stable enough to abstract over. The K8s deploy story is the next bottleneck.
3. **Kubernetes is the terminal state.** Signal: every mid-and-up team with GPU workloads either runs K8s or is migrating to K8s. Anything not K8s-native is a temporary bet.

### Why this team, why this month

Sagar has been building AI infra as a solo engineer for the last several years. The three repos going live under [github.com/plnt-work](https://github.com/plnt-work) in one session are the artifact — runtime + registry + reference SaaS, all tested, all documented, all deployable.

---

## 5. Market context

---

### Direct competitors

- **KServe** — the OSS K8s model-serving reference. General, unopinionated, supports many backends but you write InferenceService YAML per model. Great for infra-heavy teams. Wrong tool for a 4-person SaaS shop that wants a recipe-based UX.
- **NVIDIA / hyperscaler platforms** — first-party per-runtime container programs. Closed-source, hyperscaler-lock-in-shaped. plnt is the small-team, open-source, BYO-cluster alternative.

### Adjacent tools (complementary, not competitive)

- **LangChain / LangGraph / CrewAI** — Python libraries for composing agent logic. You could write a LangGraph agent, package it as a microagents recipe, deploy it with plnt. Different layer of the stack.

- **BentoML / Modal / Replicate** — API-first, hosted-first. You upload code, they run it, you pay per second. Different customer, different price model. plnt is bring-your-own-cluster.

## What we do that nobody else does

The **recipe layer**. There is no shared vocabulary today for agent workflow recipes the way Helm charts are the shared vocabulary for K8s apps. If microagents becomes that vocabulary, the runtime that ships the best pull path wins.

---

## 6. Product

---

### What ships today (v0.1)

- **Playground API** ( `plnt/playground/` ) — FastAPI, OpenAI-compat. `GET /v1/models` , `POST /v1/chat/completions` (SSE streaming), `/healthz` , `/readyz` . 15/15 contract tests passing on every commit.
- **Playground Helm chart** — Deployment, Service, ConfigMap, Ingress (SSE-safe nginx), HPA, imagePullSecrets.
- **Container image** — non-root, read-only rootfs, healthcheck. ~150 MB.
- **DigitalOcean K8s deploy overlay + cert-manager Let's Encrypt** — documented in an 11-step runbook. Cost: ~\$24/mo, single-tenant footprint.
- **Fly.io alt deploy path** — for teams that want a single-container proof surface without a full K8s cluster.
- **CLI** — `plnt playground {up, models, chat, curl}` , `plnt deploy` .
- **Site + docs** — [plnt.work](https://plnt.work), full docs portal, PRD, ERD, threat model, runbook.
- **4 GitHub repos live** under [plnt-work](https://github.com/plnt-work).

### What's next (v0.2 — 2 weeks)

- vLLM runtime chart green on a real single-GPU cluster.
- First 3 shipping recipes: `review-responder` , `post-generator` , `booking-triage` .
- Bench harness — TTFT p95, tokens/sec/GPU numbers.
- Runbook update: “adding a real model to the playground.”



## v0.3-v1.0 roadmap

- **v0.3** — TGI + SGLang runtime charts (same values contract).
- **v0.4** — Prometheus metrics + Grafana dashboard; operator wires the CRD end-to-end to the deploy saga.
- **v0.5** — TRT-LLM runtime + Triton-native adapter.
- **v1.0** — Multi-cluster failover; hosted control plane; public launch.

Full status: [github.com/plnt-work/plnt/blob/main/ROADMAP.md](https://github.com/plnt-work/plnt/blob/main/ROADMAP.md).

## 7. Architecture

### Layered view

```
+-----+
| Consumer plane                               |
| google-business SaaS | any third-party consumer |
+-----+
| API plane                                     |
| playground-api (FastAPI, OpenAI-compatible) |
| /v1/models /v1/chat/completions /healthz /readyz |
+-----+
| Runtime plane                               |
| RuntimeAdapter (mock | HTTPBackend | TritonBackend future) |
| routes on model.id                           |
+-----+
| Orchestration plane                         |
| kopf operator watches WorkflowRun CRD       |
| Temporal DeployModelWorkflow saga (5 activities) |
+-----+
| Recipe plane                               |
| microagents registry (S3 today, OCI planned) |
| workflow.yaml + prompts + tools.yaml + version + hash |
+-----+
| Cluster plane                               |
| K8s + Helm + ingress-nginx + cert-manager + HPA + GPU nodes |
+-----+
```

### Data model

The whole system has no relational database. State lives in three places:

1. Kubernetes resources — the `WorkflowRun` CRD + the standard Deployment/Service/Ingress/ConfigMap/Certificate that Helm charts render.
2. Temporal workflow state — the deploy saga's event history.
3. Playground runtime memory — the `Registry` of `RuntimeAdapter` s held in the FastAPI process, mounted from a ConfigMap at startup.

Everything is inspectable with `kubectl get events` , `helm history` , and `temporal workflow describe` . `git log` , `helm history` , and `kubectl get events` are the audit trails.

Full ERD: [github.com/plnt-work/plnt/blob/main/docs/ERD.md](https://github.com/plnt-work/plnt/blob/main/docs/ERD.md).

---

## 8. Deploy story

---

### Single-tenant DigitalOcean today

The reference deploy is DigitalOcean Kubernetes. One node (~\$12/mo), one load balancer (~\$12/mo), cert-manager for Let's Encrypt TLS, ingress-nginx for the playground route.

**Single-tenant on purpose for v0.1.** One customer per cluster keeps the security model simple (no shared-tenant blast radius) and gets us to shipping-code faster. The runbook is 11 steps, ~40 min end to end. Full recipe: [deploy/RUNBOOK-do-k8s.md](#).

### Multi-tenant later

Multi-tenant is a v1.0 concern. When it lands, the model is a shared control plane per team + isolated runtime namespaces per tenant + a per-tenant CRD scope. Not shipped, not scheduled, not committed.

### Alternate paths

- **Fly.io** — single-container proof surface with `fly deploy` . Same Dockerfile, no K8s. Good for “I just want to see the API respond over HTTPS.”
  - **Any K8s cluster** — the Helm chart is provider-agnostic. EKS / GKE / AKS / bare-metal all work.
-

## 9. Security posture

---

- Playground API pod is non-root, read-only rootfs, no privileged capabilities, all Linux capabilities dropped.
- CORS is a browser-facing allowlist, not an auth mechanism (v1 has no per-user auth by design; add Cloudflare Access if you need it).
- Container image is minimal — only playground deps. Small blast radius on CVE.
- Secrets flow via Kubernetes `Secret` refs; never inlined in Helm values.

Explicitly out of scope for v1: multi-tenant isolation, per-user auth, rate limiting, model weight provenance attestation. All in v1.0+.

Full threat model: [github.com/plnt-work/plnt/blob/main/docs/threat-model.md](https://github.com/plnt-work/plnt/blob/main/docs/threat-model.md).

---

## 10. Business model

---

### Layered pricing

- **Runtime (this repo)** — Apache-2.0, forever free. The moat is not the runtime; it is the recipe layer + hosted control plane.
- **Hosted control plane (v1.0+)** — for teams that want plnt without operating the operator themselves. Usage-based pricing: per active `WorkflowRun` per hour, or per successful invocation.
- **Enterprise support (v1.0+)** — for teams running plnt on-prem. Priority response, private-fork support, deploy help.

### Reference consumer as flywheel

`google-business` is a Google-Business SaaS built on plnt. It is both a paying customer (of itself) and the proof that the stack works end-to-end. Any Google Business owner using the SaaS is exercising the runtime in production every day.

---

## 11. Team

---

- **Sagar Bonde** — Founder. [github.com/sagarb27](https://github.com/sagarb27)

Solo for now. Playground and runtime are running against real code today. Not a slide-deck company.

---

## 12. Ask

---

Two things.

### Design partners

If you run agent workflows in production, or you're about to, we want your feedback on the recipe format. Specifically: are `workflow.yaml` + `prompts/` + `tools.yaml` enough to capture what you actually deploy, or is there a fourth thing missing?

### Seed capital (when ready)

Enough to hire two engineers — one for the reference consumer buildout, one for the hosted control plane. Runway target: 18 months. Ask: TBD when we're ready to close.

---

## 13. Links

| Where            | Link                                                                                                                      |
|------------------|---------------------------------------------------------------------------------------------------------------------------|
| Marketing        | <a href="https://plnt.work">plnt.work</a>                                                                                 |
| Playground       | <a href="https://plnt.work/playground">plnt.work/playground</a>                                                           |
| Runtime code     | <a href="https://github.com/plnt-work/plnt">github.com/plnt-work/plnt</a>                                                 |
| Recipe registry  | <a href="https://github.com/plnt-work/microagents">github.com/plnt-work/microagents</a>                                   |
| Reference SaaS   | <a href="https://github.com/plnt-work/google-business">github.com/plnt-work/google-business</a>                           |
| Site + docs      | <a href="https://github.com/plnt-work/plnt-site">github.com/plnt-work/plnt-site</a>                                       |
| Full docs portal | <a href="https://github.com/plnt-work/plnt/blob/main/docs/index.md">github.com/plnt-work/plnt/blob/main/docs/index.md</a> |

## One sentence

*plnt is the orchestration runtime for small-team agent workflows on Kubernetes.*